

An allocator-aware optional type

Abstract

Library types that can potentially hold *allocator-aware* (AA) objects should, themselves, be allocator-aware. A PMR container, for example, depends on AA types following a known AA protocol so that it (the container) can uniformly manage its memory. Even types that don't manage their own memory, such as `tuple`, follow the AA rules when they hold one more more AA elements. (A special case is `pair`, which is not actually AA but effectively follows the rules through special handling in *uses-allocator construction*.)

The current definition of `std::optional` does not follow the rules for an AA type, even when holding an AA value. This limitation makes `std::optional` unsuitable for storage in an AA container when memory allocation customization is needed.

In this paper, we propose a new, allocator-aware `std::pmr::optional` usable as a container element type and in any other context where allocator propagation is expected. This new type would not replace the current `std::optional`, but would resolve to `std::optional` when the value type is not AA. By automatically handling the distinction between AA and non-AA value types, `std::pmr::optional` can easily be used in generic code where it is not known in advance if it would be storing an AA type or not.

This is a complete proposal with formal wording. It has not yet been discussed in LEWGI or LEWG.

change history :

- replaced `alloc_optional` with additional overloads of `make_optional` that take an `allocator_arg_t` : it was observed that `alloc_optional` doesn't actually do any allocation, and that the name is not appropriate
- fixed the order of arguments in the *Returns* clause of what used to be `alloc_optional`
- adjusted for LWG 2833
- added discussion on deduction guides
- removed mentions of using a type alias since the type alias approach has proven to be problematic
- modified the specification of `std::pmr::optional` to cover both AA and non AA types
- added CTAD specification
- modified the specification of `std::pmr::optional` to be expressed in terms of `std::pmr::polymorphic_allocator<>`, as opposed to `std::pmr::memory_resource` g;

Motivation and Proposal Summary

C++ has been moving towards a consistent PMR allocator model whereby *allocator-aware* (AA) types adhere to a few rules:

1. Once constructed, an object's allocator never changes.
2. The object's constructors all follow one of two argument protocols so that they work with *uses-allocator construction* and thus can be inserted as container elements using the container's allocator.

3. An object that can contain an AA element of user-specified type should itself be AA and use its allocator to initialize its AA subobjects.

The current `std::optional` does not follow the above rules. When disengaged, it forgets its allocator, violating the first rule; its constructors don't follow the AA protocols, so containers cannot pass their allocators to optional elements; and it does not hold on an allocator by which it can initialize it's contained object. As a result, `std::optional` is not a good fit for situations where allocators are used. A `std::pmr::vector<optional<std::pmr::string>>`, for example, cannot ensure that all of strings within it use the same allocator, violating a key invariant of PMR containers. If one of the elements of the vector became disengaged and re-engaged, for example, the allocator for that one element could change. /p>

The `std::pmr::optional` class template proposed here properly adheres to the rules of an AA type when instantiated with an AA value type. It holds a copy of the allocator used to construct it, even when disengaged, and uses that allocator to construct its element when re-engaged. Containers of `std::pmr::optional` objects can correctly manage their memory.

The basic design of an AA optional is straight-forward: Add an allocator to all of its constructors and use that allocator to construct the value object each type the optional is engaged. However, when holding a non-AA type, there is no reason to pay the overhead of storing an allocator, nor to support the AA constructor interface. For this reason, `std::pmr::optional`

has a different specialisation for AA and non-AA type.

Design decisions:

Rather than support the entire C++11-style set allocators, `std::pmr::optional` works only with `std::pmr::polymorphic_allocator`. The design keeps the allocator from infecting the optional's type, in the sense that it's not a template parameter. The modern PMR-style allocator model simplifies specification, implementation, and use of `std::pmr::optional`. Allocator-aware optionals are interoperable even if they use different concrete allocators, because the concrete allocator doesn't change the type, nor does it change the size of the type.

When prototyping this proposal, we realized that an allocator-aware optional is never trivially copyable. Copy constructor and copy assignment always potentially invoke the allocator; these operations are never bitwise copies. Move operations aren't trivial either, because they may also end up doing uses-allocator construction.

This was an implementation-based reason for having an allocator-aware type separate from the usual optional. All the complexity of conditional triviality is in `std::optional`, and `std::pmr::optional`'s allocator-aware alias target has none of that.

The first proposal suggested using a type alias where a non-AA `std::pmr::optional` aliases `std::optional`, and AA `std::pmr::optional` aliases a new unnamed type. However, this causes usability issues. Type deduction in cases like :

```
template <typename T>
void foo(std::pmr::optional<T>);
```

did not work. The above was equivalent to

```
template <typename T>
void foo(std::conditional<std::uses_allocator<T, std::pmr::polymorphic_allocator<>>::value ,
    alloc_optional<T>,
    std::optional<T>>::type);
```

and using the nested type of the `std::conditional` as the function template parameter made for an undeduced context.

Feedback items for LEWG:

Should `std::pmr::optional` be placed in the `<optional>` header?

This proposal puts `std::pmr::optional` in a new header. Historically, `pmr` containers didn't have their own header files. However, `std::pmr::optional` is a more complex type than, for example, `std::pmr::vector`.

Should the deduction guide for mixed optional unpack or not?

Consider

```
std::optional x1 = value_type();
std::pmr::optional copy1 = x1; // #1

std::optional x2 = value_type_aa();
std::pmr::optional copy2 = x2; // #2

std::pmr::optional x3 = value_type();
std::optional copy3 = x3; // #3

std::pmr::optional x4 = value_type_aa();
std::optional copy4 = x4; // #4
```

What should the types of `x1`, `x2`, `x3`, and `x4` in the above example be ?

The current proposal favours unpacking, and the types are `std::pmr::optional<value_type>` , `std::pmr::optional<value_type_aa>`, `std::optional<value_type>` and `std::optional<value_type_aa>`. This seems to fit with the idea that `std::pmr::optional` and `std::optional` are types that are close enough that they deserve to be treated equally wherever they can. However, it does mean that they are somehow blessed by the standard to work together in a way a user defined type which inherits from `std::optional` wouldn't be.

Which allocator should be used in `value_or` ?

It is not all that obvious which allocator should be used for the object returned by `value_or`. Should the decision be left to the type or should it be mandated by the `optional` ? That is, should the object be constructed by plain copy/move construction or with `uses_allocator` construction?

Should assignments use `is_constructible` trait ?

It's not construct from `U` we need, it's construct from `U` using the allocator. Do we need a new trait ?

Do we need an “allocator_aware” concept?

We might consider an “allocator aware” (AA) concept that requires a `get_allocator()` method that returns something convertible to `std::pmr::polymorphic_allocator<>`. If we used this concept instead of the `std::uses_allocator` trait, the allocator would require zero extra space in most (but not all) cases. (The exception would be a type that stores its allocator outside of its footprint and whose footprint is smaller than `polymorphic_allocator<>`.)

Proposed wording

Add new paragraphs after 20.6.1/p1 [optional.general]

[2] `pmr::optional` is a class template such that for a type `T`, where `uses_allocator<T, pmr::polymorphic_allocator<>>` is true, an optional object of type `pmr::optional<T>` will construct the contained object, if any, with an allocator of type `pmr::polymorphic_allocator<>`.

Modify 20.6.2 Header synopsis1 [optional.syn].

```
namespace std {
    // [optional.optional], class template optional
    template<class T>
        class optional;
namespace pmr {
    // [pmroptional.optional], class template optional
    template<class T>
        class optional;
}
...

```

Modify 20.6.3 Class template optional [optional.optional].

```
// [optional.ctor], constructors
...
template<class U>
    explicit(see below) optional(const pmr::optional<U>&);
template<class U>
    explicit(see below) optional(pmr::optional<U>&&);
...
// [optional.assign], assignment
...
template<class U> optional& operator=(const pmr::optional&);
template<class U> optional& operator=(pmr::optional&&) noexcept(see
below);
...
template<class T>
    optional(T) -> optional<T>;
template<class T>
    optional(const pmr::optional<T>&) -> optional<T>;
template<class T>
    optional(pmr::optional<T>&&) -> optional<T>;

```

Modify 20.6.3.1 Constructors [optional.ctor].

```
template<class U> explicit(see below) optional(const optional<U>& v);
...

```

Mandates:

- is constructible `v<T, optional<U>&>` is false,
- is constructible `v<T, pmr::optional<U>&>` is false,
- is constructible `v<T, optional<U>&&>` is false,
- is constructible `v<T, pmr::optional<U>&&>` is false,
- is constructible `v<T, const optional<U>&>` is false,
- is constructible `v<T, const pmr::optional<U>&>` is false,
- is constructible `v<T, const optional<U>&&>` is false,
- is convertible `v<optional<U>&, T>` is false,
- is convertible `v<pmr::optional<U>&, T>` is false,
- is convertible `v<optional<U>&&, T>` is false,
- is convertible `v<pmr::optional<U>&&, T>` is false,
- is convertible `v<const optional<U>&, T>` is false, and
- is convertible `v<const pmr::optional<U>&, T>` is false,
- is convertible `v<const optional<U>&&, T>` is false, and
- is convertible `v<const pmr::optional<U>&&, T>` is false ...

`template<class U> explicit(see below) optional(optional<U>&& v); ...`

Mandates:

- `is constructible v<T, optional<U>&>` is false,
- `is constructible v<T, pmr::optional<U>&>` is false,
- `is constructible v<T, optional<U>&&>` is false,
- `is constructible v<T, pmr::optional<U>&&>` is false,
- `is constructible v<T, const optional<U>&>` is false,
- `is constructible v<T, const pmr::optional<U>&>` is false,
- `is constructible v<T, const optional<U>&&>` is false,
- `is constructible v<T, const pmr::optional<U>&&>` is false,
- `is convertible v<optional<U>&, T>` is false,
- `is convertible v<pmr::optional<U>&, T>` is false,
- `is convertible v<optional<U>&&, T>` is false,
- `is convertible v<pmr::optional<U>&&, T>` is false,
- `is convertible v<const optional<U>&, T>` is false, **and**
- `is convertible v<const pmr::optional<U>&, T>` is false,
- `is convertible v<const optional<U>&&, T>` is false, **and**
- `is convertible v<const pmr::optional<U>&&, T>` is false ...

...

`template<class U> explicit(see below) optional(const pmr::optional<U>& rhs);`

Effects: If rhs contains a value, initializes the contained value as if direct-non-list-initializing an object of type T with the expression `*rhs`.

Ensures: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of T.

Remarks:

The expression inside explicit is equivalent to:

`!is convertible v<const U&, T>`

Mandates:

- `is constructible v<T, const U&>` is true,
- `is same<T,U>` is false,
- `is constructible v<T, optional<U>&>` is false,
- `is constructible v<T, pmr::optional<U>&>` is false,
- `is constructible v<T, optional<U>&&>` is false,
- `is constructible v<T, pmr::optional<U>&&>` is false,
- `is constructible v<T, const optional<U>&>` is false,
- `is constructible v<T, const pmr::optional<U>&>` is false,
- `is constructible v<T, const optional<U>&&>` is false,
- `is constructible v<T, const pmr::optional<U>&&>` is false,
- `is convertible v<optional<U>&, T>` is false,
- `is convertible v<pmr::optional<U>&, T>` is false,
- `is convertible v<optional<U>&&, T>` is false,
- `is convertible v<pmr::optional<U>&&, T>` is false,
- `is convertible v<const optional<U>&, T>` is false,
- `is convertible v<const pmr::optional<U>&, T>` is false,
- `is convertible v<const optional<U>&&, T>` is false, **and**
- `is convertible v<const pmr::optional<U>&&, T>` is false

`template<class U> explicit(see below) optional(pmr::optional<U>&& rhs);`

Effects: If rhs contains a value, initializes the contained value as if direct-non-list-initializing an object of type T with the expression `std::move(*rhs)`. `std::move(*rhs)` as the constructor argument. `bool(rhs)` is unchanged.

Ensures: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of T.

Remarks: The expression inside explicit is equivalent to:

`!is convertible v<U, T>`

Mandates:

- is constructible v<T,U&&> is true,
- is same<T,U> is false,
- is constructible v<T, optional<U>&> is false,
- is constructible v<T, pmr::optional<U>&> is false,
- is constructible v<T, optional<U>&&> is false,
- is constructible v<T, pmr::optional<U>&&> is false,
- is constructible v<T, const optional<U>&> is false,
- is constructible v<T, const pmr::optional<U>&> is false,
- is constructible v<T, const optional<U>&&> is false,
- is constructible v<T, const pmr::optional<U>&&> is false,
- is convertible v<optional<U>&, T> is false,
- is convertible v<pmr::optional<U>&, T> is false,
- is convertible v<optional<U>&&, T> is false,
- is convertible v<pmr::optional<U>&&, T> is false,
- is convertible v<const optional<U>&, T> is false,
- is convertible v<const pmr::optional<U>&, T> is false,
- is convertible v<const optional<U>&&, T> is false, and
- is convertible v<const pmr::optional<U>&&, T> is false

Modify 20.6.3.3 Assignment [optional.assign].

...
 template<class U> optional& operator=(const optional<U>& rhs); ...

Remarks: ... This operator shall not participate in overload resolution unless

- is constructible v<T, const U&> is true,
- is assignable<T&, const U&> is true,
- is constructible v<T, optional<U>&> is false,
- is constructible v<T, pmr::optional<U>&> is false,
- is constructible v<T, optional<U>&&> is false,
- is constructible v<T, pmr::optional<U>&&> is false,
- is constructible v<T, const optional<U>&> is false,
- is constructible v<T, const pmr::optional<U>&> is false,
- is constructible v<T, const optional<U>&&> is false,
- is constructible v<T, const pmr::optional<U>&&> is false,
- is convertible v<optional<U>&, T> is false,
- is convertible v<pmr::optional<U>&, T> is false,
- is convertible v<optional<U>&&, T> is false,
- is convertible v<pmr::optional<U>&&, T> is false,
- is convertible v<const optional<U>&, T> is false,
- is convertible v<const pmr::optional<U>&, T> is false,
- is convertible v<const optional<U>&&, T> is false,
- is convertible v<const pmr::optional<U>&&, T> is false,
- is assignable v<T&, optional<U>&> is false,
- is assignable v<T&, pmr::optional<U>&> is false,
- is assignable v<T&, optional<U>&&> is false,
- is assignable v<T&, pmr::optional<U>&&> is false,
- is assignable v<T&, const optional<U>&> is false, and
- is assignable v<T&, const pmr::optional<U>&> is false,
- is assignable v<T&, const optional<U>&&> is false, and
- is assignable v<T&, const pmr::optional<U>&&> is false

template<class U> optional& operator=(optional<U>&& rhs); ...

Remarks: ... This operator shall not participate in overload resolution unless

- is constructible v<T, const U&> is true,
- is assignable<T&, const U&> is true,
- is constructible v<T, optional<U>&> is false,
- is constructible v<T, pmr::optional<U>&> is false,
- is constructible v<T, optional<U>&&> is false,
- is constructible v<T, pmr::optional<U>&&> is false,
- is constructible v<T, const optional<U>&> is false,
- is constructible v<T, const pmr::optional<U>&> is false,
- is constructible v<T, const optional<U>&&> is false,

- is constructible `v<T, const pmr::optional<U>&&>` is false,
- is convertible `v<optional<U>&, T>` is false,
- is convertible `v<pmr::optional<U>&, T>` is false,
- is convertible `v<optional<U>&&, T>` is false,
- is convertible `v<pmr::optional<U>&&, T>` is false,
- is convertible `v<const optional<U>&, T>` is false,
- is convertible `v<const pmr::optional<U>&, T>` is false,
- is convertible `v<const optional<U>&&, T>` is false,
- is convertible `v<const pmr::optional<U>&&, T>` is false,
- is assignable `v<T&, optional<U>&>` is false,
- is assignable `v<T&, pmr::optional<U>&>` is false,
- is assignable `v<T&, optional<U>&&>` is false,
- is assignable `v<T&, pmr::optional<U>&&>` is false,
- is assignable `v<T&, const optional<U>&>` is false, and
- is assignable `v<T&, const pmr::optional<U>&>` is false,
- is assignable `v<T&, const optional<U>&&>` is false, and
- is assignable `v<T&, const pmr::optional<U>&&>` is false

`template<class U> optional& operator=(const pmr::optional<U>& rhs);`
Effects: See Table

Table – `optional operator=(const pmr::optional<U>&)` effects

	<u>*this contains a value</u>	<u>*this does not contain a value</u>
<u>rhs contains a value</u>	<u>assigns *rhs to the contained value</u>	<u>initializes the contained value as if direct-non-list-initializing an object of type T with *rhs.</u>
<u>rhs does not contain a value</u>	<u>destroys the contained value by calling <code>val->T::~~T()</code>.</u>	<u>no effect</u>

Returns: `*this.`

Ensures: `bool(rhs) == bool(*this).`

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to T's constructor, the state of `*rhs.val` is determined by the exception safety guarantee of T's constructor. If an exception is thrown during the call to T's assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of T's assignment.

Mandates:

- is same`<T,U>` is false,
- is constructible `v<T, const U&>` is true,
- is assignable`<T&, const U&>` is true,
- is constructible `v<T, optional<U>&>` is false,
- is constructible `v<T, pmr::optional<U>&>` is false,
- is constructible `v<T, optional<U>&&>` is false,
- is constructible `v<T, pmr::optional<U>&&>` is false,
- is constructible `v<T, const optional<U>&>` is false,
- is constructible `v<T, const pmr::optional<U>&>` is false,
- is constructible `v<T, const optional<U>&&>` is false,
- is constructible `v<T, const pmr::optional<U>&&>` is false,
- is convertible `v<const U&, T>` is true,
- is convertible `v<optional<U>&, T>` is false,
- is convertible `v<pmr::optional<U>&, T>` is false,
- is convertible `v<optional<U>&&, T>` is false,
- is convertible `v<pmr::optional<U>&&, T>` is false,
- is convertible `v<const optional<U>&, T>` is false,
- is convertible `v<const pmr::optional<U>&, T>` is false,
- is convertible `v<const optional<U>&&, T>` is false,

- is convertible `v<const pmr::optional<U>&&, T>` is false,
- is assignable `v<T&, optional<U>&>` is false,
- is assignable `v<T&, pmr::optional<U>&>` is false,
- is assignable `v<T&, optional<U>&&>` is false,
- is assignable `v<T&, pmr::optional<U>&&>` is false,
- is assignable `v<T&, const optional<U>&>` is false,
- is assignable `v<T&, const pmr::optional<U>&>` is false,
- is assignable `v<T&, const optional<U>&&>` is false, and
- is assignable `v<T&, const pmr::optional<U>&&>` is false

`template<class U> optional& operator=(pmr::optional<U>&& rhs);`

Effects: See Table

Table – `optional operator=(pmr::optional<U>&&)` effects

	*this contains a value	*this does not contain a value
rhs contains a value	assigns *rhs to the contained value	initializes the contained value as if direct-non-list-initializing an object of type T with <code>std::move(*rhs)</code> .
rhs does not contain a value	destroys the contained value by calling <code>val->T::~~T()</code> .	no effect

Returns: `*this`.

Ensures: `bool(rhs) == bool(*this)`.

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to T's constructor, the state of `*rhs.val` is determined by the exception safety guarantee of T's constructor. If an exception is thrown during the call to T's assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of T's assignment.

Mandates:

- is constructible `v<T, const U&>` is true,
- is assignable `v<T&, const U&>` is true,
- is constructible `v<T, std::optional<U>&>` is false,
- is constructible `v<T, pmr::optional<U>&>` is false,
- is constructible `v<T, std::optional<U>&&>` is false,
- is constructible `v<T, pmr::optional<U>&&>` is false,
- is constructible `v<T, const std::optional<U>&>` is false,
- is constructible `v<T, const pmr::optional<U>&>` is false,
- is constructible `v<T, const std::optional<U>&&>` is false,
- is constructible `v<T, const pmr::optional<U>&&>` is false,
- is convertible `v<const U&, T>` is true,
- is convertible `v<std::optional<U>&, T>` is false,
- is convertible `v<pmr::optional<U>&, T>` is false,
- is convertible `v<std::optional<U>&&, T>` is false,
- is convertible `v<pmr::optional<U>&&, T>` is false,
- is convertible `v<const std::optional<U>&, T>` is false,
- is convertible `v<const pmr::optional<U>&, T>` is false,
- is convertible `v<const std::optional<U>&&, T>` is false,
- is convertible `v<const pmr::optional<U>&&, T>` is false,
- is assignable `v<T&, std::optional<U>&>` is false,
- is assignable `v<T&, pmr::optional<U>&>` is false,
- is assignable `v<T&, std::optional<U>&&>` is false,
- is assignable `v<T&, pmr::optional<U>&&>` is false,
- is assignable `v<T&, const std::optional<U>&>` is false,
- is assignable `v<T&, const pmr::optional<U>&>` is false,
- is assignable `v<T&, const std::optional<U>&&>` is false, and
- is assignable `v<T&, const pmr::optional<U>&&>` is false

Insert new sections after 20.6.3 Class template optional [optional.optional].

Header <pmroptional> synopsis [pmroptional.sys]

```
namespace std::pmr {
    // [pmroptional.optional]
    template<class T>
        class optional;

    // [pmroptional.traits], allocator-related traits
    template<class T>
        struct uses_allocator<T, polymorphic_allocator<>>;

    // [pmroptional.relops], relational operators
    template<class T, class U>
        bool operator==(const optional<T>&, const optional<U>&);
    template<class T, class U>
        bool operator==(const optional<T>&, const std::optional<U>&);
    template<class T, class U>
        bool operator==(const std::optional<T>&, const optional<U>&);
    template<class T, class U>
        bool operator!=(const optional<T>&, const optional<U>&);
    template<class T, class U>
        bool operator!=(const optional<T>&, const std::optional<U>&);
    template<class T, class U>
        bool operator!=(const std::optional<T>&, const optional<U>&);
    template<class T, class U>
        bool operator<(const optional<T>&, const optional<U>&);
    template<class T, class U>
        bool operator<(const optional<T>&, const std::optional<U>&);
    template<class T, class U>
        bool operator<(const std::optional<T>&, const optional<U>&);
    template<class T, class U>
        bool operator>(const optional<T>&, const optional<U>&);
    template<class T, class U>
        bool operator>(const optional<T>&, const std::optional<U>&);
    template<class T, class U>
        bool operator>(const std::optional<T>&, const optional<U>&);
    template<class T, class U>
        bool operator<=(const optional<T>&, const optional<U>&);
    template<class T, class U>
        bool operator<=(const optional<T>&, const std::optional<U>&);
    template<class T, class U>
        bool operator<=(const std::optional<T>&, const optional<U>&);
    template<class T, class U>
        bool operator>=(const optional<T>&, const optional<U>&);
    template<class T, class U>
        bool operator>=(const optional<T>&, const std::optional<U>&);
    template<class T, class U>
        bool operator>=(const std::optional<T>&, const optional<U>&);
    template<class T, three way comparable with<T> U>
        compare_three_way_result t<T,U>
            operator<=>(const optional<T>&, const optional<U>&);
    template<class T, three way comparable with<T> U>
        compare_three_way_result t<T,U>
            operator<=>(const optional<T>&, const std::optional<U>&);
    template<class T, three way comparable with<T> U>
        compare_three_way_result t<T,U>
            operator<=>(const std::optional<T>&, const optional<U>&);

    // [pmroptional.nullopts], comparison with nullopt
    template<class T> bool operator==(const optional<T>&, nullopt t)
noexcept;
    template<class T>
        strong_ordering operator<=>(const optional<T>&, nullopt t)
noexcept;
```

```

    // [pmroptional.comp.with.t], comparison with T
    template<class T, class U> bool operator==(const optional<T>&, const
U&);
    template<class T, class U> bool operator==(const T&, const
optional<U>&);
    template<class T, class U> bool operator!=(const optional<T>&, const
U&);
    template<class T, class U> bool operator!=(const T&, const
optional<U>&);
    template<class T, class U> bool operator<(const optional<T>&, const
U&);
    template<class T, class U> bool operator<(const T&, const
optional<U>&);
    template<class T, class U> bool operator>(const optional<T>&, const
U&);
    template<class T, class U> bool operator>(const T&, const
optional<U>&);
    template<class T, class U> bool operator<=(const optional<T>&, const
U&);
    template<class T, class U> bool operator<=(const T&, const
optional<U>&);
    template<class T, class U> bool operator>=(const optional<T>&, const
U&);
    template<class T, class U> bool operator>=(const T&, const
optional<U>&);
    template<class T, three way comparable with<T> U>
    compare three way result t<T,U>
    operator<=>(const optional<T>&, const U&);

// [pmroptional.specalg], specialized algorithms
template<class T>
    void swap(optional<T>&, optional<T>&) noexcept(see below);

template<class T>
    constexpr optional<see below> make optional(T&&);
template<class T>
    constexpr optional<T> make optional();
template<class T, class Arg0, class... Args>
    constexpr optional<T> make optional(Arg0 arg0&&, Args&&... args);
template<class T, class U, class... Args>
    constexpr optional<T> make optional(initializer list<U> il,
Args&&... args);

template<class T>
    constexpr optional<see below> make optional(allocator arg t,
const polymorphic allocator<>& a, T&&);
template<class T, class... Args>
    constexpr optional<T> make optional(allocator arg t, const
polymorphic allocator<>& a, Args&&... args);
template<class T, class U, class... Args>
    constexpr optional<T> make optional(allocator arg t, const
polymorphic allocator<>& a, initializer list<U> il, Args&&... args);

// [pmroptional.hash], hash support
template<class T> struct hash<optional<T>>;
}

```

Class template optional [optional.pmroptional]

```

namespace std::pmr {
    template<class T>
    class optional see below{
    public:
        using value type = T;
        using allocator type = polymorphic allocator<>; // not always

```

defined, see below

```
// [pmroptional.ctor], constructors
optional() noexcept;
optional(nullopt t) noexcept;
optional(const optional&);
optional(optional&&) noexcept(see below);
optional(const std::optional&);
optional(std::optional&&);
template<class... Args>
    explicit optional(in place t, Args&&...);
template<class U, class... Args>
    explicit optional(in place t, initializer list<U>, Args&&...);
template<class U = T>
    explicit(see below) optional(U&&);
template<class U>
    explicit(see below) optional(const optional<U>&);
template<class U>
    explicit(see below) optional(optional<U>&&);
template<class U>
    explicit(see below) optional(const std::optional<U>&);
template<class U>
    explicit(see below) optional(std::optional<U>&&);

// allocator-extended constructors
optional(allocator arg t, const polymorphic allocator<>&)
noexcept; // not always provided, see below
optional(allocator arg t, const polymorphic allocator<>&,
nullopt t) noexcept; // not always provided, see below
optional(allocator arg t, const polymorphic allocator<>&, const
optional&); // not always provided, see below
optional(allocator arg t, const polymorphic allocator<>&,
optional&&) noexcept(see below); // not always provided, see below
optional(allocator arg t, const polymorphic allocator<>&, const
std::optional&); // not always provided, see below
optional(allocator arg t, const polymorphic allocator<>&,
std::optional&&); // not always provided, see below
template<class... Args>
    explicit optional(allocator arg t, const
polymorphic allocator<>&, in place t, Args&&...); // not always
provided, see below
template<class U, class... Args>
    explicit optional(allocator arg t, const
polymorphic allocator<>&, in place t, initializer list<U>, Args&&...);
// not always provided, see below
template<class U = T>
    explicit(see below) optional(allocator arg t, const
polymorphic allocator<>&, U&&); // not always provided, see below
template<class U>
    explicit(see below) optional(allocator arg t, const
polymorphic allocator<>&, const optional<U>&); // not always provided,
see below
template<class U>
    explicit(see below) optional(allocator arg t, const
polymorphic allocator<>&, optional<U>&&); // not always provided, see
below
template<class U>
    explicit(see below) optional(allocator arg t, const
polymorphic allocator<>&, const std::optional<U>&); // not always
provided, see below
template<class U>
    explicit(see below) optional(allocator arg t, const
polymorphic allocator<>&, std::optional<U>&&); // not always provided,
see below
```

```

    // [pmroptional.dtor], destructor
    ~optional();

    // [pmroptional.assign], assignment
    optional& operator=(nullopt t) noexcept;
    optional& operator=(const optional&);
    optional& operator=(optional&&) noexcept(see below);
    template<class U = T> optional& operator=(U&&);
    template<class U> optional& operator=(const optional<U>&);
    template<class U> optional& operator=(optional<U>&&);
    template<class U> optional& operator=(const std::optional<U>&);
    template<class U> optional& operator=(std::optional<U>&&);
    template<class... Args>> T& emplace(Args&&...);
    template<class U, class... Args> T& emplace(initializer_list<U>,
Args&&...);

    // [pmroptional.swap], swap
    void swap(optional&) noexcept(see below);

    // [pmroptional.observe], observers
    const T* operator->() const;
    T* operator->();
    const T& operator*() const&;
    T& operator*() &;
    T&& operator*() &&;
    const T&& operator*() const&&;
    explicit operator bool() const noexcept;
    bool has_value() const noexcept;
    const T& value() const&;
    T& value() &;
    T&& value() &&;
    const T&& value() const&&;
    template<class U> T value or(U&&) const&;
    template<class U> T value or(U&&) &&;
    template<class U> T value or(allocator arg t, const
polymorphic allocator<>&, U&&) const&; // not always provided, see below
    template<class U> T value or(allocator arg t, const
polymorphic allocator<>&, U&&) &&; // not always provided, see below
    allocator type get_allocator() const noexcept;

    // [pmroptional.mod], modifiers
    void reset() noexcept;

private:
    T* val; // exposition only
    const polymorphic allocator<> M alloc; // exposition only; not
always provided, see below
};

template<class T>
optional(T) -> optional<T>;
template<class T>
optional(const std::optional<T>&) -> optional<T>;
template<class T>
optional(std::optional<T>&&) -> optional<T>;
}

```

Any instance of `optional<T>` at any given time either contains a value or does not contain a value. When an instance of `optional<T>` contains a value, it means that an object of type `T` is allocated within the storage of the `optional` object. Implementations are not permitted to use additional storage, such as dynamic memory, to allocate its contained value. The contained value shall be allocated in a region of the `optional<T>` storage suitably aligned for the type `T` and, if

uses_allocator<T, polymorphic_allocator<>> is true, constructed by uses_allocator construction with allocator *M_alloc*. When an object of type *optional<T>* is contextually converted to bool, the conversion returns true if the object contains a value; otherwise the conversion returns false.

Member *val* is provided for exposition only. When an *optional<T>* object contains a value, *val* points to the contained value.

If and only if uses_allocator<T, polymorphic_allocator<>> is false, *optional<T>* publicly inherits from *std::optional<T>*

If and only if uses_allocator<T, polymorphic_allocator<>> is true, the following applies :

- allocator type will be defined,
- exposition only member *M_alloc* will be provided and will be used to create the contained value, if any, and
- if <T> provides get_allocator() member function whose return type is convertible to polymorphic_allocator<>, the implementation is allowed to assume that the allocator used for constructing the element is the same allocator as returned by T::get_allocator(). [Note: In such an implementation, pmr::optional<pmr::string> will be zero-overhead compared to optional<pmr::string>. -- end note]

T shall be a type other than cv in place t, cv allocator arg t, or cv nullopt t that meets the Cpp17Destructible requirements (Table 30).

Constructors [pmroptional.ctor]

optional() noexcept;

optional(nullopt t) noexcept;

Effects: If uses_allocator<T, polymorphic_allocator<>> is true, *M_alloc* is default initialized

Ensures: *this does not contain a value.

Remarks: No contained value is initialized.

optional(const optional&);

Effects: If uses_allocator<T, polymorphic_allocator<>> is true, *M_alloc* is default initialized. If rhs contains a value, initializes the contained value with the expression *rhs.

Ensures: bool(rhs) == bool(*this).

Throws: Any exception thrown by the selected constructor of T.

Mandates: is_copyconstructible_v<T> is true.

optional(optional&& rhs) noexcept(see below)

Effects: If uses_allocator<T, polymorphic_allocator<>> is true, *M_alloc* is initialized with rhs.*M_alloc*. If rhs contains a value, initializes the contained value with the expression std::move(*rhs). bool(rhs) is unchanged.

Ensures: bool(rhs) == bool(*this).

Throws: Any exception thrown by the selected constructor of T.

Remarks: The expression inside noexcept is equivalent to is_nothrow_move_constructible_v<T>.

Mandates: is_move_constructible_v<T> is true.

optional(const std::optional&);

Effects: If uses_allocator<T, polymorphic_allocator<>> is true, *M_alloc* is default initialized. If rhs contains a value, initializes the contained value with the expression *rhs.

Ensures: bool(rhs) == bool(*this).

Throws: Any exception thrown by the selected constructor of T .

Mandates: `is_copy_constructible_v<T>` is true.

`optional(std::optional&& rhs)`

Effects: If `uses_allocator<T, polymorphic_allocator<>>` is true, `M_alloc` is initialized with `rhs.M_alloc`. If `rhs` contains a value, initializes the contained value with the expression `std::move(*rhs)`. `bool(rhs)` is unchanged.

Ensures: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of T .

Mandates: `is_move_constructible_v<T>` is true.

`template<class... Args> explicit optional(in_place_t, Args&&... args);`

Effects: If `uses_allocator<T, polymorphic_allocator<>>` is true, `M_alloc` is default initialized. Initializes the contained value with `std::forward<Args>(args).....`

Ensures: `*this` contains a value..

Throws: Any exception thrown by the selected constructor of T .

Mandates: `is_copy_constructible_v<T>` is true.

`template<class... Args> explicit optional(in_place_t, initializer_list<U> il, Args&&... args);`

Effects: If `uses_allocator<T, polymorphic_allocator<>>` is true, `M_alloc` is default initialized. Initializes the contained value with `il` and `std::forward<Args>(args).....`

Ensures: `*this` contains a value..

Throws: Any exception thrown by the selected constructor of T .

Mandates: `is_constructible_v<T, initializer_list<U>&, Args&&...>` is true.

`template<class U = T> explicit(see below) optional(U&& rhs);`

Effects: If `uses_allocator<T, polymorphic_allocator<>>` is true, `M_alloc` is default initialized. Initializes the contained value with `std::forward<U>(rhs)`.

Ensures: `*this` contains a value.

Throws: Any exception thrown by the selected constructor of T .

Remarks: The expression inside `explicit` is equivalent to:

`!is_convertible_v<U, T>`

Mandates:

- `is_constructible_v<T, U&&>` is true,
- `is_same_v<remove_cvref_t<U>, in_place_t>` is false, and
- `is_same_v<remove_cvref_t<U>, optional>` is false.

`template<class U> explicit(see below) optional(const optional<U>& rhs);`

Effects: If `uses_allocator<T, polymorphic_allocator<>>` is true, `M_alloc` is default initialized. If `rhs` contains a value, initializes the contained value with `*rhs`.

Ensures: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of T .

Remarks: The expression inside `explicit` is equivalent to:

`!is_convertible_v<const U&, T>`

Mandates:

- `is_constructible_v<T, const U&>` is true,
- `is_constructible_v<T, std::optional<U>&>` is false,
- `is_constructible_v<T, optional<U>&>` is false,
- `is_constructible_v<T, std::optional<U>&&>` is false,
- `is_constructible_v<T, optional<U>&&>` is false,
- `is_constructible_v<T, const std::optional<U>&>` is false,

- is_constructible_v<T, const optional<U>&> is false,
- is_constructible_v<T, const std::optional<U>&&> is false,
- is_constructible_v<T, const optional<U>&&> is false,
- is_convertible_v<std::optional<U>&, T> is false,
- is_convertible_v<optional<U>&, T> is false,
- is_convertible_v<std::optional<U>&&, T> is false,
- is_convertible_v<optional<U>&&, T> is false,
- is_convertible_v<const std::optional<U>&, T> is false,
- is_convertible_v<const optional<U>&, T> is false,
- is_convertible_v<const std::optional<U>&&, T> is false, and
- is_convertible_v<const optional<U>&&, T> is false

template<class U> explicit(see below) optional(optional<U>&& rhs);

Effects: If uses_allocator<T, polymorphic_allocator<>> is true, *M_alloc* is default initialized. If rhs contains a value, initializes the contained value with std::move(*rhs). bool(rhs) is unchanged.

Ensures: bool(rhs) == bool(*this).

Throws: Any exception thrown by the selected constructor of T.

Remarks: The expression inside explicit is equivalent to:

- !is_convertible_v<U, T> *Mandates:*
- is_constructible_v<T, U&&> is true,
 - is_constructible_v<T, std::optional<U>&> is false,
 - is_constructible_v<T, optional<U>&> is false,
 - is_constructible_v<T, std::optional<U>&&> is false,
 - is_constructible_v<T, optional<U>&&> is false,
 - is_constructible_v<T, const std::optional<U>&> is false,
 - is_constructible_v<T, const optional<U>&> is false,
 - is_constructible_v<T, const std::optional<U>&&> is false,
 - is_constructible_v<T, const optional<U>&&> is false,
 - is_convertible_v<std::optional<U>&, T> is false,
 - is_convertible_v<optional<U>&, T> is false,
 - is_convertible_v<std::optional<U>&&, T> is false,
 - is_convertible_v<optional<U>&&, T> is false,
 - is_convertible_v<const std::optional<U>&, T> is false,
 - is_convertible_v<const optional<U>&, T> is false,
 - is_convertible_v<const std::optional<U>&&, T> is false, and
 - is_convertible_v<const optional<U>&&, T> is false

template<class U> explicit(see below) optional(const std::optional<U>& rhs);

Effects: If uses_allocator<T, polymorphic_allocator<>> is true, *M_alloc* is default initialized. If rhs contains a value, initializes the contained value with *rhs.

Ensures: bool(rhs) == bool(*this).

Throws: Any exception thrown by the selected constructor of T.

Remarks: The expression inside explicit is equivalent to:

- !is_convertible_v<const U&, T>

Mandates:

- is_constructible_v<T, const U&> is true,
- is_constructible_v<T, std::optional<U>&> is false,
- is_constructible_v<T, optional<U>&> is false,
- is_constructible_v<T, std::optional<U>&&> is false,
- is_constructible_v<T, optional<U>&&> is false,
- is_constructible_v<T, const std::optional<U>&> is false,

- `is_constructible_v<T, const optional<U>&>` is false,
- `is_constructible_v<T, const std::optional<U>&&>` is false,
- `is_constructible_v<T, const optional<U>&&>` is false,
- `is_convertible_v<std::optional<U>&, T>` is false,
- `is_convertible_v<optional<U>&, T>` is false,
- `is_convertible_v<std::optional<U>&&, T>` is false,
- `is_convertible_v<optional<U>&&, T>` is false,
- `is_convertible_v<const std::optional<U>&, T>` is false,
- `is_convertible_v<const optional<U>&, T>` is false,
- `is_convertible_v<const std::optional<U>&&, T>` is false, and
- `is_convertible_v<const optional<U>&&, T>` is false

`template<class U> explicit(see below) optional(std::optional<U>&& rhs);`

Effects: If `uses_allocator<T, polymorphic_allocator<>>` is true, `M_alloc` is default initialized. If `rhs` contains a value, initializes the contained value with `std::move(*rhs)`. `bool(rhs)` is unchanged.

Ensures: `bool(rhs) == bool(*this)`.

Throws: Any exception thrown by the selected constructor of `T`.

Remarks:

The expression inside `explicit` is equivalent to:

`!is_convertible_v<U, T>`

Mandates:

- `is_constructible_v<T, U&&>` is true,
- `is_constructible_v<T, std::optional<U>&>` is false,
- `is_constructible_v<T, optional<U>&>` is false,
- `is_constructible_v<T, std::optional<U>&&>` is false,
- `is_constructible_v<T, optional<U>&&>` is false,
- `is_constructible_v<T, const std::optional<U>&>` is false,
- `is_constructible_v<T, const optional<U>&>` is false,
- `is_constructible_v<T, const std::optional<U>&&>` is false,
- `is_constructible_v<T, const optional<U>&&>` is false,
- `is_convertible_v<std::optional<U>&, T>` is false,
- `is_convertible_v<optional<U>&, T>` is false,
- `is_convertible_v<std::optional<U>&&, T>` is false,
- `is_convertible_v<optional<U>&&, T>` is false,
- `is_convertible_v<const std::optional<U>&, T>` is false,
- `is_convertible_v<const optional<U>&, T>` is false,
- `is_convertible_v<const std::optional<U>&&, T>` is false, and
- `is_convertible_v<const optional<U>&&, T>` is false

```
optional(allocator_arg_t, const polymorphic_allocator<>&) noexcept;
optional(allocator_arg_t, const polymorphic_allocator<>&, nullopt_t)
noexcept;
optional(allocator_arg_t, const polymorphic_allocator<>&, const
optional&);
optional(allocator_arg_t, const polymorphic_allocator<>&, optional&&)
noexcept(see below);
optional(allocator_arg_t, const polymorphic_allocator<>&, const
std::optional&);
optional(allocator_arg_t, const polymorphic_allocator<>&,
std::optional&&);
template<class... Args>
explicit optional(allocator_arg_t, const polymorphic_allocator<>&,
in_place_t, Args&&...);
template<class U, class... Args>
```

```

    explicit optional(allocator_arg_t, const polymorphic_allocator<>&,
in_place_t, initializer_list<U>, Args&&...);
template<class U = T>
    explicit(see below) optional(allocator_arg_t, const
polymorphic_allocator<>&, U&&);
template<class U>
    explicit(see below) optional(allocator_arg_t, const
polymorphic_allocator<>&, const optional<U>&);
template<class U>
    explicit(see below) optional(allocator_arg_t, const
polymorphic_allocator<>&, optional<U>&&);
template<class U>
    explicit(see below) optional(allocator_arg_t, const
polymorphic_allocator<>&, const std::optional<U>&);
template<class U>
    explicit(see below) optional(allocator_arg_t, const
polymorphic_allocator<>&, std::optional<U>&&);
    Constraints: uses_allocator<T, polymorphic_allocator<>> is true;
    Effects: Equivalent to the preceding constructors except that M_alloc is initialized
with a.

```

Destructor [pmroptional.dtor]

```

~optional().;
    Effects: If is trivially destructible  $v<T> \neq \text{true}$  and *this contains a
value, calls  $\text{val} \rightarrow T::\sim T()$ .
    Remarks: If is trivially destructible  $v<T>$  is true, then this destructor is
trivial.

```

Assignment [pmroptional.assign]

```

optional<T>& operator=(nullopt t) noexcept;
    Effects: If *this contains a value, calls  $\text{val} \rightarrow T::\sim T()$  to destroy the contained
value; otherwise no effect.
    Remarks: *this
    Ensures: *this does not contain a value.

optional<T>& operator=(const optional& rhs);
    Effects: See Table

```

Table — optional operator=(const optional& rhs) effects

	<u>*this contains a value</u>	<u>*this does not contain a value</u>
<u>rhs contains a value</u>	<u>assigns *rhs to the contained value</u>	<u>initializes the contained value with *rhs. If uses_allocator<T, polymorphic_allocator<>> is true, initialization is done by uses-allocator construction with allocator <i>M_alloc</i></u>
<u>rhs does not contain a value</u>	<u>destroys the contained value by calling $\text{val} \rightarrow T::\sim T()$</u>	<u>no effect</u>

Returns: `*this`.

Ensures: `bool(rhs) == bool(*this)`.

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s copy constructor, no effect. If an exception is thrown during the call to `T`'s copy assignment, the state of its contained value is as defined by the exception safety guarantee of `T`'s copy assignment. This operator shall be defined as deleted unless `is_copy_constructible_v<T>` is true and `is_copy_assignable_v<T>` is true.

`optional<T>& operator=(optional&& rhs) noexcept(see below);`

Effects: See Table

Table — `optional operator=(optional&& rhs)` effects

	<code>*this</code> contains a value	<code>*this</code> does not contain a value
<code>rhs</code> contains a value	assigns <code>std::move(*rhs)</code> to the contained value	initializes the contained value with <code>std::move(*rhs)</code> . If <code>uses_allocator<T, polymorphic_allocator<>></code> is true, initialization is done by <code>uses-allocator</code> construction with allocator <code>M alloc</code>
<code>rhs</code> does not contain a value	destroys the contained value by calling <code>val->T::~T()</code>	no effect

Returns: `*this`.

Ensures: `bool(rhs) == bool(*this)`.

Remarks: The expression inside `noexcept` is equivalent to:

`is_nothrow_move_assignable_v<T> &&
is_nothrow_move_constructible_v<T>`

If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s move constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `T`'s move constructor. If an exception is thrown during the call to `T`'s move assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `T`'s move assignment.

Mandates: `is_move_constructible_v<T>` is true and `is_move_assignable_v<T>` is true.

`template<class U = T> optional& operator=(U&& v);`

Effects: If `*this` contains a value, assigns `std::forward<U>(v)` to the contained value; otherwise initializes the contained value with `std::forward<U>(v)`. If `uses_allocator<T, polymorphic_allocator<>>` is true, initialization is done by `uses-allocator` construction with allocator `M alloc`

Returns: `*this`.

Ensures: `*this` contains a value.

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `v` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `*val` and `v` is determined by the exception safety guarantee of `T`'s assignment.

Mandates:

- is same $v<\text{remove cvref } t<U>, \text{ alloc-optional}>$ is false,
- conjunction $v<\text{is scalar}<T>, \text{ is same}<T, \text{ decay } t<U>>>$ is false,
- is constructible $v<T, U>$ is true, and
- is assignable $v<T\&, U>$ is true

template<class U> optional& operator=(const optional<U>& rhs);

Effects: See Table

Table — optional operator=(const optional<U>&) effects

	<i>*this</i> <i>contains a</i> <i>value</i>	<i>*this does not contain a value</i>
<i>rhs</i> <i>contains a</i> <i>value</i>	<i>assigns *rhs</i> <i>to the</i> <i>contained</i> <i>value</i>	<i>initializes the contained value with *rhs. If</i> <i>uses allocator<T,</i> <i>polymorphic allocator<>> is true,</i> <i>initialization is done by uses-allocator</i> <i>construction with allocator M alloc</i>
<i>rhs does</i> <i>not</i> <i>contain a</i> <i>value</i>	<i>destroys the</i> <i>contained</i> <i>value by</i> <i>calling val-</i> <i>>T::~T(.)</i>	<i>no effect</i>

Returns: **this*.

Ensures: $\text{bool}(\text{rhs}) == \text{bool}(*\text{this})$.

Remarks: If any exception is thrown, the result of the expression $\text{bool}(*\text{this})$ remains unchanged. If an exception is thrown during the call to T 's constructor, no effect. If an exception is thrown during the call to T 's assignment, the state of **val* is determined by the exception safety guarantee of T 's assignment.

Mandates:

- is same $v<T, U>$ is false,
- is constructible $v<T, \text{ const } U\&>$ is true,
- is assignable $v<T\&, \text{ const } U\&>$ is true,
- is constructible $v<T, \text{ std}::\text{optional}<U>\&>$ is false,
- is constructible $v<T, \text{ optional}<U>\&>$ is false,
- is constructible $v<T, \text{ std}::\text{optional}<U>\&\&>$ is false,
- is constructible $v<T, \text{ optional}<U>\&\&>$ is false,
- is constructible $v<T, \text{ const std}::\text{optional}<U>\&>$ is false,
- is constructible $v<T, \text{ const optional}<U>\&>$ is false,
- is constructible $v<T, \text{ const std}::\text{optional}<U>\&\&>$ is false,
- is constructible $v<T, \text{ const optional}<U>\&\&>$ is false,
- is convertible $v<\text{std}::\text{optional}<U>\&, T>$ is false,
- is convertible $v<\text{optional}<U>\&, T>$ is false,
- is convertible $v<\text{std}::\text{optional}<U>\&\&, T>$ is false,
- is convertible $v<\text{optional}<U>\&\&, T>$ is false,
- is convertible $v<\text{const std}::\text{optional}<U>\&, T>$ is false,
- is convertible $v<\text{const optional}<U>\&, T>$ is false,
- is convertible $v<\text{const std}::\text{optional}<U>\&\&, T>$ is false,
- is convertible $v<\text{const optional}<U>\&\&, T>$ is false,
- is assignable $v<T\&, \text{ std}::\text{optional}<U>\&>$ is false,
- is assignable $v<T\&, \text{ optional}<U>\&>$ is false,
- is assignable $v<T\&, \text{ std}::\text{optional}<U>\&\&>$ is false,
- is assignable $v<T\&, \text{ optional}<U>\&\&>$ is false,

- is assignable $v<T\&, \text{const std::optional}<U>\&\>$ is false,
- is assignable $v<T\&, \text{const optional}<U>\&\>$ is false,
- is assignable $v<T\&, \text{const std::optional}<U>\&\&\>$ is false, and
- is assignable $v<T\&, \text{const optional}<U>\&\&\>$ is false

template<class U> optional& operator=(const std::optional<U>& rhs);

Effects: The result of the expression `bool(rhs)` remains unchanged. See Table

Table — optional operator=(const std::optional<U>&) effects

	*this contains a value	*this does not contain a value
rhs contains a value	assigns <code>std::move(*rhs)</code> to the contained value	initializes the contained value with <code>std::move(*rhs)</code> . If <code>uses_allocator<T, polymorphic_allocator<>></code> is true, initialization is done by uses-allocator construction with allocator <code>M alloc</code> .
rhs does not contain a value	destroys the contained value by calling <code>val->T::~T()</code> .	no effect

Returns: `*this`.

Ensures: `bool(rhs) == bool(*this)`.

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, no effect. If an exception is thrown during the call to `T`'s assignment, the state of `*val` is determined by the exception safety guarantee of `T`'s assignment.

Mandates:

- is constructible $v<T, U>$ is true,
- is assignable $<T\&, U>$ is true,
- is constructible $v<T, \text{std::optional}<U>\&\>$ is false,
- is constructible $v<T, \text{optional}<U>\&\>$ is false,
- is constructible $v<T, \text{std::optional}<U>\&\&\>$ is false,
- is constructible $v<T, \text{optional}<U>\&\&\>$ is false,
- is constructible $v<T, \text{const std::optional}<U>\&\>$ is false,
- is constructible $v<T, \text{const optional}<U>\&\>$ is false,
- is constructible $v<T, \text{const std::optional}<U>\&\&\>$ is false,
- is constructible $v<T, \text{const optional}<U>\&\&\>$ is false,
- is convertible $v<\text{std::optional}<U>\&, T>$ is false,
- is convertible $v<\text{optional}<U>\&, T>$ is false,
- is convertible $v<\text{std::optional}<U>\&\&, T>$ is false,
- is convertible $v<\text{optional}<U>\&\&, T>$ is false,
- is convertible $v<\text{const std::optional}<U>\&, T>$ is false,
- is convertible $v<\text{const optional}<U>\&, T>$ is false,
- is convertible $v<\text{const std::optional}<U>\&\&, T>$ is false,
- is convertible $v<\text{const optional}<U>\&\&, T>$ is false,
- is assignable $v<T\&, \text{std::optional}<U>\&\>$ is false,
- is assignable $v<T\&, \text{optional}<U>\&\>$ is false,
- is assignable $v<T\&, \text{std::optional}<U>\&\&\>$ is false,
- is assignable $v<T\&, \text{optional}<U>\&\&\>$ is false,
- is assignable $v<T\&, \text{const std::optional}<U>\&\>$ is false,
- is assignable $v<T\&, \text{const optional}<U>\&\>$ is false,

- is assignable $v < T\&$, $\text{const std::optional}<U>\&\&$ is false, and

- is assignable $v < T\&$, $\text{const optional}<U>\&\&$ is false

`template<class U> optional& operator=(optional<U>&& rhs);`

Effects: See Table

Table — `optional operator=(optional<U>&&)` effects

	<u><code>*this</code> <code>contains a</code> <code>value</code></u>	<u><code>*this</code> does not contain a value</u>
<u><code>rhs</code> <code>contains a</code> <code>value</code></u>	<u><code>assigns *rhs</code> <code>to the</code> <code>contained</code> <code>value</code></u>	<u><code>initializes the contained value with *rhs. If</code> <code>uses allocator<T,</code> <code>polymorphic allocator<>> is true,</code> <code>initialization is done by uses-allocator</code> <code>construction with allocator M alloc .</code></u>
<u><code>rhs does</code> <code>not</code> <code>contain a</code> <code>value</code></u>	<u><code>destroys the</code> <code>contained</code> <code>value by</code> <code>calling val-</code> <code>>T::~~T(.)</code></u>	<u><code>no effect</code></u>

Returns: `*this`.

Ensures: `bool(rhs) == bool(*this)`.

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s move constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `T`'s move constructor. If an exception is thrown during the call to `T`'s move assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `T`'s move assignment.

Mandates:

- is same $<T, U>$ is false,

- is constructible $v < T$, $\text{const } U\&$ is true,

- is assignable $<T\&$, $\text{const } U\&$ is true,

- is constructible $v < T$, $\text{std::optional}<U>\&$ is false,

- is constructible $v < T$, $\text{optional}<U>\&$ is false,

- is constructible $v < T$, $\text{std::optional}<U>\&\&$ is false,

- is constructible $v < T$, $\text{optional}<U>\&\&$ is false,

- is constructible $v < T$, $\text{const std::optional}<U>\&$ is false,

- is constructible $v < T$, $\text{const optional}<U>\&$ is false,

- is constructible $v < T$, $\text{const std::optional}<U>\&\&$ is false,

- is constructible $v < T$, $\text{const optional}<U>\&\&$ is false,

- is convertible $v < \text{const } U\&$, T is true,

- is convertible $v < \text{std::optional}<U>\&$, T is false,

- is convertible $v < \text{optional}<U>\&$, T is false,

- is convertible $v < \text{std::optional}<U>\&\&$, T is false,

- is convertible $v < \text{optional}<U>\&\&$, T is false,

- is convertible $v < \text{const std::optional}<U>\&$, T is false,

- is convertible $v < \text{const optional}<U>\&$, T is false,

- is convertible $v < \text{const std::optional}<U>\&\&$, T is false,

- is convertible $v < \text{const optional}<U>\&\&$, T is false,

- is assignable $v < T\&$, $\text{std::optional}<U>\&$ is false,

- is assignable $v < T\&$, $\text{optional}<U>\&$ is false,

- is assignable $v < T\&$, $\text{std::optional}<U>\&\&$ is false,

- is assignable $v<T\&, \text{optional}<U>\&\&>$ is false,
- is assignable $v<T\&, \text{const std::optional}<U>\&>$ is false,
- is assignable $v<T\&, \text{const optional}<U>\&>$ is false,
- is assignable $v<T\&, \text{const std::optional}<U>\&\&>$ is false, and
- is assignable $v<T\&, \text{const optional}<U>\&\&>$ is false

`template<class U> optional& operator=(std::optional<U>&& rhs);`

Effects: See Table

Table — optional operator=(std::optional<U>&&) effects

	<u>*this contains a value</u>	<u>*this does not contain a value</u>
<u>rhs contains a value</u>	<u>assigns *rhs to the contained value</u>	<u>initializes the contained value with *rhs. If uses allocator<T>, polymorphic allocator<>> is true, initialization is done by uses-allocator construction with allocator M alloc</u>
<u>rhs does not contain a value</u>	<u>destroys the contained value by calling val- >T::~T(.)</u>	<u>no effect</u>

Returns: *this.

Ensures: `bool(rhs) == bool(*this)`.

Remarks: If any exception is thrown, the result of the expression `bool(*this)` remains unchanged. If an exception is thrown during the call to `T`'s constructor, the state of `*rhs.val` is determined by the exception safety guarantee of `T`'s constructor. If an exception is thrown during the call to `T`'s assignment, the state of `*val` and `*rhs.val` is determined by the exception safety guarantee of `T`'s assignment.

Mandates:

- is constructible $v<T, \text{const } U\&>$ is true,
- is assignable $v<T\&, \text{const } U\&>$ is true,
- is constructible $v<T, \text{std::optional}<U>\&>$ is false,
- is constructible $v<T, \text{optional}<U>\&>$ is false,
- is constructible $v<T, \text{std::optional}<U>\&\&>$ is false,
- is constructible $v<T, \text{optional}<U>\&\&>$ is false,
- is constructible $v<T, \text{const std::optional}<U>\&>$ is false,
- is constructible $v<T, \text{const optional}<U>\&>$ is false,
- is constructible $v<T, \text{const std::optional}<U>\&\&>$ is false,
- is constructible $v<T, \text{const optional}<U>\&\&>$ is false,
- is convertible $v<\text{const } U\&, T>$ is true,
- is convertible $v<\text{std::optional}<U>\&, T>$ is false,
- is convertible $v<\text{optional}<U>\&, T>$ is false,
- is convertible $v<\text{std::optional}<U>\&\&, T>$ is false,
- is convertible $v<\text{optional}<U>\&\&, T>$ is false,
- is convertible $v<\text{const std::optional}<U>\&, T>$ is false,
- is convertible $v<\text{const optional}<U>\&, T>$ is false,
- is convertible $v<\text{const std::optional}<U>\&\&, T>$ is false,
- is convertible $v<\text{const optional}<U>\&\&, T>$ is false,
- is assignable $v<T\&, \text{std::optional}<U>\&>$ is false,
- is assignable $v<T\&, \text{optional}<U>\&>$ is false,

- is assignable $v < T\&$, $\text{std}::\text{optional}<U>\&\&$ is false,
- is assignable $v < T\&$, $\text{optional}<U>\&\&$ is false,
- is assignable $v < T\&$, $\text{const std}::\text{optional}<U>\&$ is false,
- is assignable $v < T\&$, $\text{const optional}<U>\&$ is false,
- is assignable $v < T\&$, $\text{const std}::\text{optional}<U>\&\&$ is false, and
- is assignable $v < T\&$, $\text{const optional}<U>\&\&$ is false

`template<class... Args>> T& emplace(Args&&...);`

Constraints: is constructible $v < T$, $\text{Args}\&\&\dots$ is true.

Effects: Calls $*this = \text{nullptr}$. Then initializes the contained value with $\text{std}::\text{forward}<\text{Args}>(\text{args})\dots$. If uses $\text{allocator}<T, \text{polymorphic allocator}<>>$ is true, initialization is done by uses-allocator construction with allocator M alloc.

Ensures: $*this$ contains a value.

Returns: A reference to the new contained value.

Throws: Any exception thrown by the selected constructor of T .

Remarks: If an exception is thrown during the call to T 's constructor, $*this$ does not contain a value, and the previous $*val$ (if any) has been destroyed.

`template<class U, class... Args> T& emplace(initializer_list<U>, Args&&...);`

Constraints: is constructible $v < T$, $\text{initializer_list}<U>\&$, $\text{Args}\&\&\dots$ is true.

Effects: Calls $*this = \text{nullptr}$. Then initializes the contained value with il and $\text{std}::\text{forward}<\text{Args}>(\text{args})\dots$. If uses $\text{allocator}<T, \text{polymorphic allocator}<>>$ is true, initialization is done by uses-allocator construction with allocator M alloc.

Ensures: $*this$ contains a value.

Returns: A reference to the new contained value.

Throws: Any exception thrown by the selected constructor of T .

Remarks: If an exception is thrown during the call to T 's constructor, $*this$ does not contain a value, and the previous $*val$ (if any) has been destroyed.

Swap [pmroptional.swap]

`void swap(optional& rhs);`

Constraints: Lvalues of type T shall be swappable and is move constructible - $v < T$ is true.

Expects: If uses $\text{allocator}<T, \text{polymorphic allocator}<>>$ is true, $a.\text{get_allocator}() == b.\text{get_allocator}()$.

Effects: See Table x

Table x — `swap(optional&)` effects

	$*this$ contains a value	$*this$ does not contain a value
rhs contains a value	<code>calls swap(*(*this), *rhs)</code>	initializes the contained value with <code>std::move(*rhs)</code> as the constructor argument, followed by <code>rhs.val->T::~T()</code> ; If uses $\text{allocator}<T, \text{polymorphic allocator}<>>$ is true, initialization is done by uses-allocator construction with allocator M alloc. Postcondition is that $*this$

		<u>contains a value and rhs does not contain a value</u>
<u>rhs does not contain a value</u>	<u>initializes the contained value with <code>std::move>(*this)</code> as the constructor argument, followed by <code>val->T::~T()</code>. If <code>uses_allocator<T,</code> <u>polymorphic_allocator<>> is true</u>, initialization is done by <code>uses-allocator</code> construction with allocator <code>M alloc</code>. <u>Postcondition is that <code>*this</code> does not contain a value and <code>rhs</code> contains a value</u></u>	<u>no effect</u>

Throws: Any exceptions thrown by the operations in the relevant part of Table x.

Observers [pmroptional.observe]

```
const T* operator->() const;  
T* operator->();
```

Expects: `*this` contains a value.

Returns: `val`.

Throws: Nothing.

```
const T& operator*() const&;  
T& operator*() &;().;
```

Expects: `*this` contains a value.

Returns: `*val`.

Throws: Nothing.

```
T&& operator*() &&;  
const T&& operator*() const&&; &;().;
```

Expects: `*this` contains a value.

Effects: Equivalent to: `return std::move(*val);`

```
explicit operator bool() const noexcept;
```

Returns: true if and only if contains a value.

```
bool has_value() const noexcept;
```

Returns: true if and only if contains a value.

```
const T& value() const&;  
T& value() &;
```

Effects: Equivalent to:

`return bool(*this) ? *val : throw bad_optional_access();`

```
T&& value() &&;  
const T&& value() const&&;
```

Effects: Equivalent to:

`return bool(*this) ? std::move(*val) : throw bad_optional_access();`

```
template<class U> T value or(U&& v) const&;
```

Effects: Equivalent to:

```
return bool(*this) ? **this : static cast<T>(std::forward<U>(v)).;
```

Remarks: If *is copy constructible* *v<T> && is convertible v<U&&, T> is false, the program is ill-formed.*

```
template<class U> T value or(U&& v) &&;
```

Effects: Equivalent to:

```
return bool(*this) ? std::move(**this) : static cast<T>(std::forward<U>(v)).;
```

Remarks: If *is move constructible* *v<T> && is convertible v<U&&, T> is false, the program is ill-formed.*

```
template<class U> T value or(allocator arg t, const polymorphic allocator<>& a, U&& v) const&tt>
```

Constraints: *uses allocator<T, polymorphic allocator<>> is true;*

Effects: if **this* contains a value, returns an object of type *T* initialized by *uses-allocator* construction with allocator *a* and ***this* as the constructor argument. Otherwise, returns an object of type *T* initialized by *uses-allocator* construction with allocator *a* and *std::forward<U>(v)* as the constructor argument.

Remarks: If *is copy constructible* *v<T> && is convertible v<U&&, T> is false, the program is ill-formed.*

```
template<class U> T value or(allocator arg t, const polymorphic allocator<>& a, U&& v) &&;
```

Constraints: *uses allocator<T, polymorphic allocator<>> is true;*

Effects: if **this* contains a value, returns an object of type *T* initialized by *uses-allocator* construction with allocator *a* and *std::move(**this)* as the constructor argument. Otherwise, returns an object of type *T* initialized by *uses-allocator* construction with allocator *a* and *std::forward<U>(v)* as the constructor argument.

Remarks: If *is copy constructible* *v<T> && is convertible v<U&&, T> is false, the program is ill-formed.*

```
allocator type get allocator() const noexcept;
```

Constraints: *uses allocator<T, polymorphic allocator<>> is true;*

Returns: *M alloc.*

Throws: *nothing.*

Modifiers [pmroptional.mod]

```
void reset() noexcept;
```

Effects: If **this* contains a value, calls *val->T::~T()* to destroy the contained value; otherwise no effect.

Ensures: **this* does not contain a value.

Allocator-related traits [pmroptional.traits]

```
template<class T> struct uses_allocator<T, polymorphic_allocator<>> : uses_allocator<T, polymorphic_allocator<>> {_};
```

Relational operators [pmroptional.relops]

```
template<class T, class U> bool operator==(const optional<T>&, const
```

```

optional<U>&);
    template<class T, class U> bool operator==(const optional<T>&, const
std::optional<U>&);
    template<class T, class U> bool operator==(const std::optional<T>&,
const optional<U>&);
    Mandates: The expression *x == *y shall be well-formed and its result
shall be convertible to bool. [ Note: T need not be
Cpp17EqualityComparable. – end note ]
    Returns: If bool(x) != bool(y), false; otherwise if bool(x) == false,
true; otherwise *x == *y.

    template<class T, class U> bool operator!=(const optional<T>&, const
optional<U>&);
    template<class T, class U> bool operator!=(const optional<T>&, const
std::optional<U>&);
    template<class T, class U> bool operator!=(const std::optional<T>&,
const optional<U>&);
    Mandates: The expression *x != *y shall be well-formed and its result
shall be convertible to bool.
    Returns: If bool(x) != bool(y), true; otherwise if bool(x) == false,
false; otherwise *x != *y.

    template<class T, class U> bool operator<(const optional<T>&, const
optional<U>&);
    template<class T, class U> bool operator<(const optional<T>&, const
std::optional<U>&);
    template<class T, class U> bool operator<(const std::optional<T>&,
const optional<U>&);
    Mandates: The expression *x < *y shall be well-formed and its result
shall be convertible to bool.
    Returns: If !y, false; otherwise if !x, true; otherwise *x < *y.

    template<class T, class U> bool operator>(const optional<T>&, const
optional<U>&);
    template<class T, class U> bool operator>(const optional<T>&, const
std::optional<U>&);
    template<class T, class U> bool operator>(const std::optional<T>&,
const optional<U>&);
    Mandates: The expression *x > *y shall be well-formed and its result
shall be convertible to bool.
    Returns: If !x, false; otherwise if !y, true; otherwise *x > *y.

    template<class T, class U> bool operator<=(const optional<T>&, const
optional<U>&);
    template<class T, class U> bool operator<=(const optional<T>&, const
std::optional<U>&);
    template<class T, class U> bool operator<=(const std::optional<T>&,
const optional<U>&);
    Mandates: The expression *x <= *y shall be well-formed and its result
shall be convertible to bool.
    Returns: If !x, true; otherwise if !y, false; otherwise *x <= *y.

    template<class T, class U> bool operator&t;=(const optional<T>&,
const optional<U>&);
    template<class T, class U> bool operator&t;=(const optional<T>&, const
std::optional<U>&);
    template<class T, class U> bool operator&t;=(const std::optional<T>&,
const optional<U>&);
    Mandates: The expression *x &t;= *y shall be well-formed and its result
shall be convertible to bool.
    Returns: If !y, true; otherwise if !x, false; otherwise *x &t;= *y.

    template<class T, three way comparable with<T> U>
    compare three way result t<T,U>
    operator<=>(const optional<T>&, const optional<U>&);

```

```

template<class T, three way comparable with<T> U>
    compare three way result t<T,U>
        operator<=>(const optional<T>&, const std::optional<U>&);
template<class T, three way comparable with<T> U>
    compare three way result t<T,U>
        operator<=>(const std::optional<T>&, const optional<U>&);
Returns: If x && y, *x <=> *y; otherwise bool(x) <=> bool(y).

```

Comparison with nullopt [pmoptional.nullopts]

```

template<class T> bool operator==(const optional<T>&, nullopt t)
noexcept;
Returns: !x.

```

```

template<class T> strong ordering operator<=>(const optional<T>&,
nullopt t) noexcept;
Returns: bool(x) <=> false.

```

Comparison with T [pmoptional.comp.with.t]

```

template<class T, class U> bool operator==(const optional<T>& x,
const U& v);
Mandates: The expression *x == v shall be well-formed and its result
shall be convertible to bool. [ Note: T need not be
Cpp17EqualityComparable. – end note ]
Effects: Equivalent to: return bool(x) ? *x == v : false;

```

```

template<class T, class U> bool operator==(const T& t, const
optional<U>& v);
Mandates: The expression x == *v shall be well-formed and its result
shall be convertible to bool.
Effects: Equivalent to: return bool(v) ? x == *v : false;

```

```

template<class T, class U> bool operator!=(const optional<T>& u,
const U& v);
Mandates: The expression *x != v shall be well-formed and its result
shall be convertible to bool.
Effects: Equivalent to: return bool(x) ? *x != v : true;

```

```

template<class T, class U> bool operator!=(const T& t, const
optional<U>& v);
Mandates: The expression x != *v shall be well-formed and its result
shall be convertible to bool.
Effects: Equivalent to: return bool(v) ? x != *v : true;

```

```

template<class T, class U> bool operator<(const optional<T>&, const
U&);
Mandates: The expression *x < v shall be well-formed and its result
shall be convertible to bool.
Effects: Equivalent to: return bool(x) ? *x < v : true;

```

```

template<class T, class U> bool operator<(const T& t, const
optional<U>& v);
Mandates: The expression x < *v shall be well-formed and its result
shall be convertible to bool.
Effects: Equivalent to: return bool(v) ? x < *v : false;

```

```

template<class T, class U> bool operator>(const optional<T>&, const
U&);
Mandates: The expression *x > v shall be well-formed and its result
shall be convertible to bool.
Effects: Equivalent to: return bool(x) ? *x > v : false;

```

template<class T, class U> bool operator>(const T& t, const optional<U>& v);

Mandates: The expression $x > *v$ shall be well-formed and its result shall be convertible to bool.

Effects: Equivalent to: $\text{return bool}(v) ? x > *v : \text{true};$

template<class T, class U> bool operator<=(const optional<T>&, const U&);

Mandates: The expression $*x \leq v$ shall be well-formed and its result shall be convertible to bool.

Effects: Equivalent to: $\text{return bool}(x) ? *x \leq v : \text{true};$

template<class T, class U> bool operator<=(const T& t, const optional<U>& v);

Mandates: The expression $x \leq *v$ shall be well-formed and its result shall be convertible to bool.

Effects: Equivalent to: $\text{return bool}(v) ? x \leq *v : \text{false};$

template<class T, class U> bool operator>=(const optional<T>&, const U&);

Mandates: The expression $*x \geq v$ shall be well-formed and its result shall be convertible to bool.

Effects: Equivalent to: $\text{return bool}(x) ? *x \geq v : \text{false};$

template<class T, class U> bool operator>=(const T& t, const optional<U>& v);

Mandates: The expression $x \geq *v$ shall be well-formed and its result shall be convertible to bool.

Effects: Equivalent to: $\text{return bool}(v) ? x \geq *v : \text{true};$

template<class T, three way comparable with<T> U>

compare three way result t<T,U>

operator<=>(const optional<T>& x, const U&v);

Effects: Equivalent to: $\text{return bool}(x) ? *x \geq v : \text{strong ordering::less};$

Specialized algorithms [pmroptional.specalg]

template<class T> void swap(optional<T>& x, optional<T>& y)
noexcept(see below);

Effects: Calls $x.\text{swap}(y)$.

Remarks: This function shall not participate in overload resolution unless $\text{is_move_constructible } v<T>$ is true and $\text{is_swappable } v<T>$ is true.

template<class T> constexpr optional<decay t<T>> make optional(T&&);

Returns: $\text{optional}<\text{decay } t<T>>(\text{std}::\text{forward}<T>(v))$.

Remarks: Specialization of this function template shall be a constexpr function if $\text{uses_allocator}<T, \text{polymorphic_allocator}<>>::\text{value}$ is false.

template<class T> constexpr optional<T> make optional();

Effects: Equivalent to: $\text{return optional}<T>(\text{in_place})$.

Remarks: Specialization of this function template shall be a constexpr function if $\text{uses_allocator}<T, \text{polymorphic_allocator}<>>::\text{value}$ is false.

template<class T, class Arg0, class... Args> constexpr optional<T>
make optional(Arg0&& arg0, Args&&... args);

Effects: Equivalent to: $\text{return optional}<T>(\text{in_place}, \text{std}::\text{forward}<Arg0>(arg0), \text{std}::\text{forward}<Args>(args) \dots)$.

Constraints: $! \text{is_same}<\text{typename } \text{remove_cvref } t<Arg0>, \text{allocator } arg_t>$

Remarks: Specialization of this function template shall be a constexpr function if $\text{uses_allocator}<T, \text{polymorphic_allocator}<>>::\text{value}$

is false.

```
template<class T, class... Args> constexpr optional<T>
make_optional(Args&&... args);
Effects: Equivalent to: return optional<T>(in_place, std::
forward<Args>(args)...).
Remarks: Specialization of this function template shall be a
constexpr function if uses allocator<T, polymorphic allocator<>>::value
is false.
```

```
template<class T, class U, class... Args> optional<T>
constexpr make_optional(initializer_list<U> il, Args&&... args);
Effects: Equivalent to: return optional<T>(in_place, il, std::
forward<Args>(args)...).
Remarks: Specialization of this function template shall be a
constexpr function if uses allocator<T, polymorphic allocator<>>::value
is false.
```

```
template<class T, optional<decay_t<T>>
make_optional(allocator arg_t, const polymorphic allocator<>& a,
T&&);
Mandates: uses allocator<T, polymorphic allocator<>> is true;
Returns: optional<decay_t<T>>(allocator arg, a, in_place, std::
forward<T>(v)).
```

```
template<class T, class... Args> optional<T>
make_optional(allocator arg_t, const polymorphic allocator<>& a,
Args&&... args);
Mandates: uses allocator<T, polymorphic allocator<>> is true;
Effects: Equivalent to: return optional<T>(allocator arg, a,
in_place, std::forward<Args>(args)...).
```

```
template<class T, class U, class... Args> optional<T>
make_optional(allocator arg_t, const polymorphic allocator<>& a,
initializer_list<U> il, Args&&... args);
Mandates: uses allocator<T, polymorphic allocator<>> is true;
Effects: Equivalent to: return optional<T>(allocator arg, a,
in_place, il, std::forward<Args>(args)...).
```

Hash support [pmroptional.hash]

```
template<class T> struct hash<optional<T>>;
The specialization hash<optional<T>> is enabled ([unord.hash]) if and
only if hash<remove_const_t<T>> is enabled. When enabled, for an object
o of type optional<T>, if bool(o) == true, then hash<optional<T>>()(o)
shall evaluate to the same value as hash<remove_const_t<T>>()(o);
otherwise it evaluates to an unspecified value. The member functions are
not guaranteed to be noexcept.
```